



รายงานปฏิบัติการ PS06: Virtual Memory

เสนอ

อาจารย์ ราชวิทย์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	1
1 ข้อ 1: ทำความเข้าใจแนวคิดของ Virtual Memory	1
1.1 หลักการพื้นฐานที่ระบบปฏิบัติการสมัยใหม่ใช้ร่วมกัน	1
1.2 การ implement บน Linux	2
1.3 การ implement บน MS Windows	2
1.4 ตารางเปรียบเทียบ Linux กับ MS Windows	3
1.5 ขั้นตอนการเพิ่มและลด swap space บน Linux	4
1.5.1 ขั้นตอนที่ 1: ตรวจสอบสถานะ swap เดิมก่อนแก้ไข	4
1.5.2 ขั้นตอนที่ 2: สร้าง file สำหรับใช้เป็น swap	4
1.5.3 ขั้นตอนที่ 3: ตั้งสิทธิ์และ format พื้นที่ให้เป็น swap area	5
1.5.4 ขั้นตอนที่ 4: เปิดใช้งาน swap file	5
1.5.5 ขั้นตอนที่ 5: ยืนยันว่า swap เพิ่มขึ้นจริง	5
1.5.6 ขั้นตอนที่ 6: ทำให้ถาวร และข้อจำกัดที่พบบน WSL2	6
1.5.7 ขั้นตอนที่ 7: ลด swap space	7
1.5.8 ขั้นตอนที่ 8: ลบ file และคืนค่าระบบให้เหมือนเดิม	7
1.5.9 การปรับพฤติกรรมการใช้ swap ด้วย vm.swappiness	7
1.5.10 สรุปคำสั่งทั้งหมดของข้อ 1.2	8
2 ข้อ 2: การเฝ้าดู Virtual Memory บน Linux	8
2.1 การรัน free และผลลัพธ์ที่ได้	8
2.1.1 ความหมายของทุกฟิลด์ที่คำสั่งนี้แสดง	9
2.1.2 สรุปตัวเลขทั้งหมดที่คำสั่ง free รายงาน	9
2.2 argument ที่น่าสนใจของ free	10
2.2.1 argument ที่ 1: -w ร่วมกับ -t	10
2.2.2 argument ที่ 2: -s ร่วมกับ -c	11
2.3 การติดตั้งและรัน vmstat	11
2.3.1 ความหมายของทุกคอลัมน์ที่คำสั่งนี้แสดง	12
2.3.2 สรุปตัวเลขทั้งหมดที่คำสั่ง vmstat รายงาน	12
2.4 argument ที่น่าสนใจของ vmstat	13
2.4.1 argument ที่ 1: -s m ร่วมกับการระบุ interval และ count	13
2.4.2 argument ที่ 2: -s ร่วมกับ -s m	13
2.5 ความสอดคล้องระหว่างตัวเลขของ free และ vmstat	14
3 ข้อ 3: การเฝ้าดู Virtual Memory บน MS Windows	15
3.1 การเปิด Resource Monitor	15
3.2 ข้อมูลที่แสดงในแท็บ Memory	15
3.2.1 ส่วนที่ 1: ตาราง Processes	15
3.2.2 ส่วนที่ 2: แท็บ Physical Memory	16
3.2.3 ส่วนที่ 3: กราฟด้านขวา	16
3.3 สรุปตัวเลขทั้งหมดในส่วน Physical Memory	16
3.4 การเทียบศัพท์ระหว่าง Windows กับ Linux	17
สรุปผลการทดลอง	18

เอกสารอ้างอิง

19

สภาพแวดล้อมที่ใช้ในการทดลอง

ปฏิบัติการนี้แบ่งการทดลองออกเป็นสองฝั่ง คือฝั่ง Linux (ข้อ 1 และข้อ 2) และฝั่ง MS Windows (ข้อ 3) โดยทั้งสองฝั่งทำงานอยู่บนเครื่องคอมพิวเตอร์เดียวกัน รายละเอียดของสภาพแวดล้อมเป็นดังนี้

Table 1: ข้อมูลสภาพแวดล้อมของระบบที่ใช้ในการทดลอง

รายการ	รายละเอียด
ระบบปฏิบัติการ ฝั่ง Windows	Windows 11 Home Single Language (build 26200)
ระบบปฏิบัติการฝั่ง Linux Kernel ของ Linux	Ubuntu 24.04.1 LTS ทำงานบน WSL2 6.6.87.2-microsoft-standard-WSL2 (x86-64)
หน่วยความจำหลักที่เห็น	15 GiB (WSL2 แบ่งมาจาก RAM ของเครื่อง)
Swap เดิมของระบบ Linux init ของ Linux	4 GiB เป็น partition /dev/sdc ที่ WSL2 สร้างให้อัตโนมัติ systemd (เปิดผ่าน systemd=true ใน /etc/wsl.conf)
เครื่องมือ monitoring	procps-ng 4.0.4 ซึ่งเป็น package ที่ให้ทั้ง free และ vmstat
เครื่องมือฝั่ง Windows	Resource Monitor (resmon.exe) ที่มากับระบบปฏิบัติการ

หมายเหตุเรื่องหลักฐาน: เนื่องจากใบงานกำหนดให้แสดงภาพหน้าจอของผลการรันคำสั่ง (ข้อ 1.2 ระบุ *with screenshots* และข้อ 2 ระบุ *Please capture the output image from running the command*) รายงานฉบับนี้จึงใช้ภาพหน้าจอจริงเป็นหลักฐานของทุกขั้นตอน ตัวเลขทุกตัวที่อ้างถึงในคำอธิบายและย่อหน้าสรุปอ่านมาจากภาพที่แสดงไว้ในรายงานโดยตรง ภาพทั้งหมดเก็บไว้ที่ PS06/result/screenshots/ และลำดับคำสั่งที่ใช้ถ่ายภาพอยู่ใน PS06/docs/COMMANDS.md

1 ข้อ 1: ทำความเข้าใจแนวคิดของ Virtual Memory

1.1 หลักการพื้นฐานที่ระบบปฏิบัติการสมัยใหม่ใช้ร่วมกัน

Virtual Memory คือกลไกที่ระบบปฏิบัติการสร้าง address space เสมือนขึ้นมาให้แต่ละ process โดย address ที่ program มองเห็น (virtual address) ไม่ใช่ address จริงบนแผง RAM (physical address) แต่ต้องผ่านการแปลงด้วย hardware หน่วยหนึ่งชื่อ MMU (Memory Management Unit) เสียก่อน แนวคิดนี้ให้ประโยชน์สามข้อพร้อมกัน

- **การแยกขอบเขต (Isolation):** แต่ละ process มีตาราง mapping ของตัวเอง process หนึ่งจึงอ้างถึงหน่วยความจำของอีก process หนึ่งไม่ได้ เป็นพื้นฐานของความปลอดภัยและความเสถียรของระบบ
- **การใช้หน่วยความจำเกินขนาดจริง (Over-subscription):** ผลรวมของ address space ของทุก process ใหญ่กว่า RAM จริงได้ เพราะ page ที่ไม่ได้ใช้งานจะถูกย้ายไปเก็บบน disk (swap หรือ page file)
- **การแบ่งใช้หน่วยความจำร่วมกัน (Sharing):** page เดียวกันบน RAM ถูก map เข้าไปในหลาย process ได้ เช่น code ของ shared library หรือ page ที่เกิดจาก fork() ซึ่งใช้เทคนิค copy-on-write

หน่วยที่เล็กที่สุดของการจัดการคือ **page** (ฝั่ง virtual) และ **page frame** (ฝั่ง physical) ซึ่งบนสถาปัตยกรรม x86-64 มีขนาดมาตรฐาน 4 KiB การแปลง address ใช้โครงสร้างที่เรียกว่า page table ซึ่งบน x86-64 เป็นแบบ 4 ระดับ (CPU รุ่นใหม่บางรุ่นรองรับ 5 ระดับ) และเพื่อไม่ให้ต้องเดิน page table ทุกครั้งทีอ่านเขียนหน่วยความจำ CPU จึง cache ผลการแปลงล่าสุดไว้ใน TLB (Translation Lookaside Buffer)

เมื่อ process อ้างถึง page ที่ยังไม่มี mapping ที่ใช้งานได้ CPU จะยกข้อผิดพลาดที่เรียกว่า **page fault** ขึ้นมาให้ kernel จัดการ แบ่งเป็นสองแบบสำคัญ

- **Soft fault (minor fault):** ข้อมูลอยู่ใน RAM อยู่แล้ว เพียงแต่ยังไม่ได้ map เข้ากับ process นั้น เช่น page อยู่ใน page cache ของ Linux หรืออยู่ใน standby list ของ Windows ก็ได้เร็วมากเพราะไม่ต้องอ่าน disk
- **Hard fault (major fault):** ข้อมูลไม่อยู่ใน RAM ต้องอ่านกลับจาก disk ซึ่งช้ากว่าหลายหมื่นเท่า จำนวน hard fault ต่อวินาทีจึงเป็นตัวชี้วัดสำคัญว่าเครื่องขาดหน่วยความจำจริงหรือไม่

เทคนิคอื่นที่ทั้งสองระบบปฏิบัติการใช้เหมือนกันคือ **demand paging** (โหลด page เข้ามาเมื่อถูกอ้างถึงจริงเท่านั้น), **copy-on-write** (สำเนา page ต่อเมื่อมีการเขียนจริง), **memory-mapped file** (map file เข้ากับ address space โดยตรง) และ **page replacement** (เลือก page ที่จะถูกไล่ออกจาก RAM เมื่อหน่วยความจำใกล้หมด) หากหน่วยความจำไม่พอจนระบบต้องสลับ page เข้าออก disk ตลอดเวลาจนแทบไม่ได้ทำงานจริง เรียกอาการนั้นว่า **thrashing**

1.2 การ implement บน Linux

โครงสร้างข้อมูลหลัก ทุก process ใน Linux มี `struct mm_struct` หนึ่งชุดเป็นตัวแทนของ address space ภายในเก็บรายการของ `struct vm_area_struct` (เรียกสั้น ๆ ว่า VMA) ซึ่งแต่ละตัวแทน region ที่มีคุณสมบัติเหมือนกันหนึ่งช่วง เช่น code segment, heap, stack หรือ region ที่เกิดจาก `mmap()` ส่วน page frame ทุก frame บนเครื่องมี `struct page` กำกับไว้หนึ่งตัวเพื่อเก็บสถานะ เช่น ตัวบ่งการอ้างอิงและ flag ว่า page นั้นสกปรก (dirty) หรือไม่มีการแปลง address ใช้ page table 4 ระดับคือ PGD, PUD, PMD และ PTE

ประเภทของ page Linux แบ่ง page ที่เรียกคืนได้ออกเป็นสองกลุ่มใหญ่ คือ **file-backed page** ซึ่งมีต้นฉบับอยู่บน file system อยู่แล้ว (page cache) เรียกคืนได้โดยเขียนกลับเฉพาะ page ที่ dirty กับ **anonymous page** ซึ่งไม่มีต้นฉบับบน disk เช่น heap และ stack การจะเรียกคืน anonymous page ได้ต้องเขียนลง swap ก่อนเสมอ นี่คือเหตุผลที่ระบบซึ่งไม่มี swap เลยจะเรียกคืนได้เฉพาะ page cache เท่านั้น

การเรียกคืนหน่วยความจำ Linux จัดกลุ่ม page ไว้ในรายการแบบ LRU สองชั้นคือ active list และ inactive list แยกกันระหว่าง anonymous และ file-backed เมื่อหน่วยความจำว่างต่ำกว่า watermark ที่กำหนด kernel thread ชื่อ `kswapd` จะตื่นขึ้นมาเรียกคืน page ในเบื้องหลัง และถ้าหน่วยความจำหมดเร็วกว่าที่ `kswapd` ตามทัน process ที่ขอหน่วยความจำจะต้องช่วยเรียกคืนเองแบบ direct reclaim ซึ่งทำให้ process นั้นหยุดรอ ค่า `vm.swappiness` (0–200 ค่าเริ่มต้นคือ 60) เป็นตัวกำหนดว่า kernel จะเอนไปเรียกคืน anonymous page ผ่าน swap มากน้อยเพียงใดเมื่อเทียบกับการทิ้ง page cache

นโยบาย overcommit และ OOM killer โดยค่าเริ่มต้น Linux ยอมให้ process จองหน่วยความจำรวมกันเกินขนาดจริงได้ (`vm.overcommit_memory = 0`) เพราะโดยทั่วไป program จองมากกว่าที่ใช้จริง ผลคือ `malloc()` มักสำเร็จเสมอ แต่ถ้าถึงจุดที่ทุก process เขียนข้อมูลลงหน่วยความจำที่จองไว้จนเต็มเครื่องจริง kernel จะเรียก **OOM killer** ขึ้นมาเลือกฆ่า process ที่มีคะแนน `oom_score` สูงที่สุดทิ้ง ซึ่งต่างจาก Windows ที่ใช้วิธีปฏิเสธการจองตั้งแต่แรก

รูปแบบของ swap Linux รองรับทั้ง swap partition และ swap file โดยประสิทธิภาพต่างกันน้อยมากบน file system สมัยใหม่ ทำให้ swap file เป็นที่นิยมกว่าเพราะปรับขนาดได้โดยไม่ต้องแบ่ง partition ใหม่ นอกจากนี้ยังมี `zram` (block device สำหรับ swap ที่บีบอัดข้อมูลไว้ใน RAM) และ `zswap` (cache แบบบีบอัดที่คั่นอยู่หน้า swap จริง) ซึ่งทำหน้าที่คล้ายกับ Memory Compression ของ Windows รวมถึง Transparent Huge Pages ที่รวม page ขนาด 4 KiB จำนวน 512 หน้าให้เป็น page ขนาด 2 MiB เพื่อลด TLB miss

1.3 การ implement บน MS Windows

โครงสร้างข้อมูลหลัก ฝั่ง Windows ใช้ **VAD tree** (Virtual Address Descriptor) ซึ่งเป็น balanced tree เก็บ region ของ address space ของแต่ละ process แทนรายการ VMA ของ Linux ส่วนสถานะของ page frame ทุก frame เก็บรวมศูนย์ไว้ในโครงสร้างชื่อ **PFN database** การแปลง address ก็ใช้ page table 4 ระดับเช่นกันเพราะ

ทำงานบน hardware ชุดเดียวกัน

Working set และการจัดสรรแบบรายบ้าน แนวคิดที่ต่างจาก Linux ชัดที่สุดคือ Windows จัดการหน่วยความจำเป็น **working set** ของแต่ละ process กล่าวคือแต่ละ process มีโควตา page ที่ถือครองอยู่ใน RAM ได้ และมี component ชื่อ **Balance Set Manager** คอยตัดแต่งโควตานี้ (working set trimming) เมื่อระบบขาดหน่วยความจำ ขณะที่ Linux มองหน่วยความจำเป็น pool รวมของทั้งระบบแล้วเรียกคืนตาม LRU ของทั้งเครื่อง

รายการ page ระดับระบบ page ที่ถูกตัดออกจาก working set จะไม่ถูกทิ้งทันที แต่ไหลเข้าสู่รายการกลางสามรายการ ได้แก่ **modified list** (page ที่ถูกแก้ไขแล้ว รอ Modified Page Writer เขียนลง page file หรือ file ดัชนี), **standby list** (page ที่สะอาดแล้ว ยังเก็บเนื้อหาไว้ ใช้เป็น cache ของทั้งระบบ) และ **free/zero list** (page ว่างพร้อมแจกจ่าย) จุดสำคัญคือ page ที่อยู่ใน standby list ยังดึงกลับเข้า working set ได้ด้วย soft fault ซึ่งเร็วมาก บทบาทของ standby list จึงเทียบได้กับ page cache ของ Linux และเป็นเหตุผลที่ Resource Monitor แสดงหน่วยความจำส่วนนี้แยกออกมาให้เห็นในข้อ 3

Commit charge และ page file Windows ไม่ใช้นโยบาย overcommit แบบ Linux แต่ใช้ระบบ **commit charge** คือทุกครั้งที่ process ขอหน่วยความจำแบบ private ระบบจะจองโควตาไว้ในบัญชีกลางทันที และ **commit limit** ของระบบเท่ากับขนาด RAM รวมกับขนาดของ `pagefile.sys` เมื่อ commit charge ขนพาดาน การจองครั้งถัดไปจะล้มเหลวทันทีพร้อมข้อความแจ้งเตือนหน่วยความจำต่ำ แทนที่จะฆ่า process ทิ้งแบบ OOM killer โดยค่าเริ่มต้น Windows ให้ระบบจัดการขนาดของ `pagefile.sys` เองและขยายได้อัตโนมัติ

กลไกเสริม Windows 10 ขึ้นไปมี **Memory Compression** ซึ่งบีบอัด page ที่กำลังจะถูกย้ายลง page file เก็บไว้ใน RAM ก่อน เพื่อลดการอ่านเขียน disk และมี **SysMain** (ชื่อเดิมคือ Superfetch) ที่เรียนรู้รูปแบบการใช้งานแล้วโหลด page ของ program ที่ผู้ใช้เปิดบ่อยเข้า standby list ไว้ล่วงหน้า

1.4 ตารางเปรียบเทียบ Linux กับ MS Windows

Table 2: เปรียบเทียบการ implement Virtual Memory ระหว่าง Linux และ MS Windows

ประเด็น	Linux	MS Windows
โครงสร้างของ address space	mm_struct และรายการ VMA	VAD tree ต่อหนึ่ง process
สถานะของ page frame	struct page ต่อหนึ่ง frame	PFN database รวมศูนย์
หน่วยของการเรียกคืน	LRU active/inactive list ของทั้งระบบ	working set รายบ้าน ตัดแต่งโดย Balance Set Manager
ผู้ทำงานเบื้องหลัง	kswapd และ direct reclaim	Balance Set Manager และ Modified Page Writer
ที่พัก page ก่อนถูกทิ้ง	page cache และ inactive list	modified list, standby list, free/zero list
Backing store	swap partition หรือ swap file มีได้หลายชุด และกำหนด priority ได้	<code>pagefile.sys</code> ปกติหนึ่งชุด ให้ระบบจัดการขนาดเอง
การบีบอัดหน่วยความจำ	zram หรือ zswap ต้องเปิดใช้งานเอง	Memory Compression เปิดใช้งานโดยค่าเริ่มต้น
นโยบายการจอง	overcommit ได้ตาม <code>vm.overcommit_memory</code>	commit charge เทียบกับ commit limit ที่เท่ากับ RAM บวกขนาด page file
เมื่อหน่วยความจำหมด	OOM killer เลือกฆ่า process ที่ oom_score สูงสุด	ปฏิเสธการจองครั้งใหม่และแจ้งเตือน low memory
ปุ่มปรับพฤติกรรม	<code>vm.swappiness</code> , <code>vm.vfs_cache_pressure</code> , <code>vm.overcommit_ratio</code>	ตั้งขนาด page file ผ่าน System Properties หรือ registry
เครื่องมือ monitoring มาตรฐาน	<code>free</code> , <code>vmstat</code> , <code>/proc/meminfo</code> , <code>top</code>	Task Manager, Resource Monitor, Performance Monitor

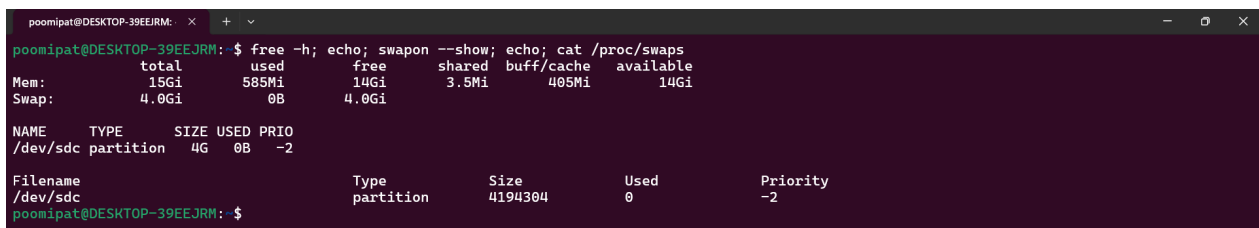
สรุปความต่างที่สำคัญที่สุด ทั้งสองระบบใช้ hardware และหลักการเดียวกันคือ paging กับ demand paging แต่ต่างกันที่ มุมมองในการจัดการ คือ Linux มองหน่วยความจำเป็น pool กลางของทั้งเครื่องแล้วเรียกคืนตามอายุการใช้งานของ page ส่วน Windows มองเป็นโคตารายบ้านของแต่ละ process แล้วค่อยรวบรวม page ที่ตัดออกมาไว้ในรายการกลาง และต่างกันที่ นโยบายการจอง คือ Linux ยอมให้จองเกินแล้วไปเสี่ยงเอาตอนใช้งานจริง ส่วน Windows กันโควตาไว้ตั้งแต่ตอนจองจึงไม่จำเป็นต้องฆ่า process ทิ้งภายหลัง

1.5 ขั้นตอนการเพิ่มและลด swap space บน Linux

หัวข้อนี้สาธิตการเพิ่มและลด swap space จริงบน Ubuntu 24.04.1 ที่ทำงานบน WSL2 โดยเลือกใช้วิธี swap file แทนการแบ่ง swap partition ใหม่ ด้วยเหตุผลสองข้อคือ swap file สร้างและลบได้ทันทีโดยไม่ต้องยุ่งกับตาราง partition ของ disk และบน file system สมัยใหม่ประสิทธิภาพต่างจาก swap partition น้อยมาก ทุกขั้นตอนถ่ายภาพหน้าจอไว้เป็นหลักฐานตามลำดับการทำงานจริง

1.5.1 ขั้นตอนที่ 1: ตรวจสอบสถานะ swap เดิมก่อนแก้ไข

ก่อนแก้ไขสิ่งใดต้องรู้สถานะตั้งต้นเสมอ คำสั่งที่ใช้คือ free -h เพื่อดูภาพรวม swapon --show เพื่อดูว่ามี swap area ไตเปิดใช้งานอยู่บ้าง และ cat /proc/swaps ซึ่งเป็นแหล่งข้อมูลดิบที่ kernel เปิดให้อ่าน



```
poomipat@DESKTOP-39EEJRM: ~$ free -h; echo; swapon --show; echo; cat /proc/swaps
              total        used        free      shared  buff/cache   available
Mem:            15Gi         585Mi          14Gi          3.5Mi          405Mi          14Gi
Swap:            4.0Gi           0B           4.0Gi

NAME      TYPE      SIZE USED PRIO
/dev/sdc  partition  4G    0B   -2

Filename      Type      Size      Used      Priority
/dev/sdc      partition  4194304      0        -2
poomipat@DESKTOP-39EEJRM: ~$
```

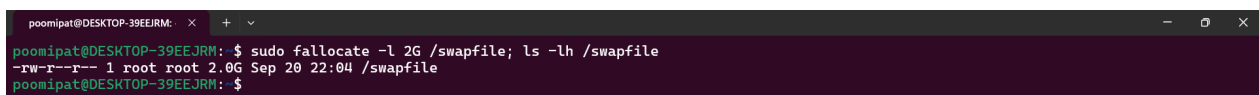
Figure 1: สถานะตั้งต้น: swap เดิมมีเพียง partition /dev/sdc ขนาด 4 GiB

จากภาพ ระบบมีหน่วยความจำหลัก 15 GiB ใช้ไป 585 MiB และมี swap ทั้งหมด 4.0 GiB ที่ยังไม่ถูกใช้เลย (used เท่ากับ 0B) โดย swap นี้เป็นชนิด partition ชื่อ /dev/sdc มีค่า priority เท่ากับ -2 ส่วน /proc/swaps รายงานขนาดเป็นหน่วย KiB คือ 4194304 KiB ซึ่งเท่ากับ 4 GiB พอดี จุดที่ควรสังเกตคือ swap ก่อนนี้ไม่ได้เกิดจากการติดตั้ง Ubuntu เอง แต่เป็น virtual disk ที่ WSL2 สร้างให้อัตโนมัติ (ขนาดเริ่มต้นเท่ากับ 25% ของ RAM ที่ virtual machine มองเห็น)

1.5.2 ขั้นตอนที่ 2: สร้าง file สำหรับใช้เป็น swap

```
sudo fallocate -l 2G /swapfile
ls -lh /swapfile
```

Code 1: สร้าง file ขนาด 2 GiB ด้วย fallocate



```
poomipat@DESKTOP-39EEJRM: ~$ sudo fallocate -l 2G /swapfile; ls -lh /swapfile
-rw-r--r-- 1 root root 2.0G Sep 20 22:04 /swapfile
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 2: fallocate -l 2G จองพื้นที่บน disk ให้ครบ 2 GiB ในคำสั่งเดียว

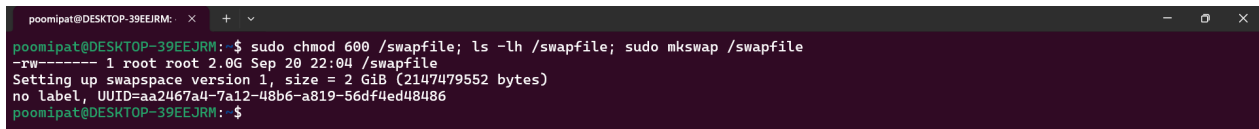
fallocate จองพื้นที่บน file system ให้ครบตามขนาดที่ระบุทันทีโดยไม่ต้องเขียนข้อมูลลงไปจริง จึงเร็วกว่าการใช้ dd มาก ผลที่ได้คือ file /swapfile ขนาด 2.0G เจ้าของเป็น root เพราะสร้างผ่าน sudo แต่สิทธิ์ที่ได้คือ -rw-r--r-- (644) ซึ่งหมายความว่าผู้ใช้ทั่วไปอ่าน file นี้ได้ ถือเป็นช่องโหว่ร้ายแรงเพราะ swap เก็บข้อมูลที่ถูกลบออกจากหน่วยความจำของทุก process รวมถึงข้อมูลที่เป็นความลับ จึงต้องแก้สิทธิ์ในขั้นถัดไปก่อนนำไปใช้งาน

ทางเลือก: หาก file system ไม่รองรับ `fallocate` หรือ `swapon` ปฏิเสธ file ที่มีช่องว่าง (ข้อความ it appears to have holes) ให้สร้างด้วยการเขียนข้อมูลจริงแทน คือ `sudo dd if=/dev/zero of=/swapfile bs=1M count=2048`

1.5.3 ขั้นที่ 3: ตั้งสิทธิ์และ format พื้นที่ให้เป็น swap area

```
sudo chmod 600 /swapfile
ls -lh /swapfile
sudo mkswap /swapfile
```

Code 2: ปิดสิทธิ์ให้เหลือเฉพาะ root แล้วเขียน header ของ swap



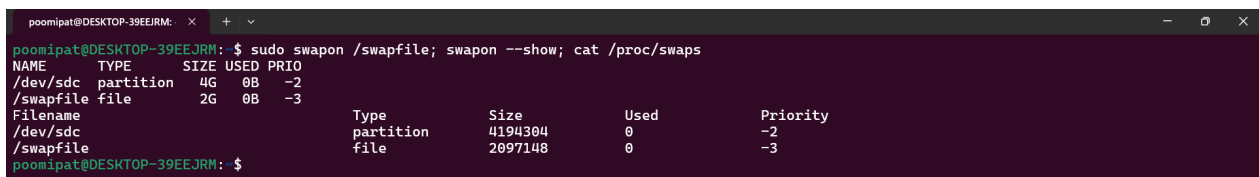
```
poomipat@DESKTOP-39EEJRM: ~$ sudo chmod 600 /swapfile; ls -lh /swapfile; sudo mkswap /swapfile
-rw----- 1 root root 2.0G Sep 20 22:04 /swapfile
Setting up swapspace version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=aa2467a4-7a12-48b6-a819-56df4ed48486
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 3: หลัง `chmod 600` สิทธิ์เหลือเฉพาะ root และ `mkswap` เขียน header พร้อม UUID

สิทธิ์เปลี่ยนเป็น `-rw-----` คือมีเพียง root เท่านั้นที่อ่านหรือเขียนได้ จากนั้น `mkswap` จะเขียนโครงสร้าง header ของ swap area ลงไปที่ต้นของ file แล้วรายงานว่า Setting up swapspace version 1, size = 2 GiB (2147479552 bytes) พร้อมกำหนด UUID ให้หนึ่งค่า

ตัวเลขที่น่าสนใจคือ 2147479552 bytes ซึ่งน้อยกว่า 2 GiB (2147483648 bytes) อยู่ 4096 bytes พอตี ส่วนที่หายไปคือ page แรกของ file ที่ถูกกันไว้เก็บ header ของ swap area จึงเหลือพื้นที่ใช้งานจริง 2097148 KiB ตามที่จะเห็นใน `/proc/swaps` ในขั้นถัดไป

1.5.4 ขั้นที่ 4: เปิดใช้งาน swap file



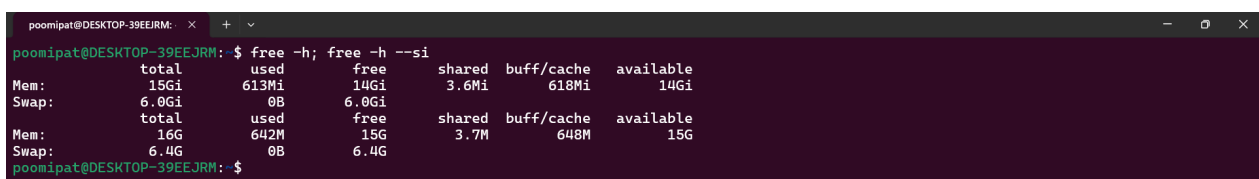
```
poomipat@DESKTOP-39EEJRM: ~$ sudo swapon /swapfile; swapon --show; cat /proc/swaps
NAME      TYPE      SIZE USED PRIO
/dev/sdc  partition 4G    0B   -2
/swapfile file       2G    0B   -3
Filename  Type      Size      Used      Priority
/dev/sdc  partition 4194304    0         -2
/swapfile file      2097148    0         -3
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 4: หลัง `swapon` ระบบมี swap area สองก้อนทำงานพร้อมกัน

หลังสั่ง `sudo swapon /swapfile` ระบบมี swap area สองก้อนคือ partition `/dev/sdc` ขนาด 4G และ file `/swapfile` ขนาด 2G ทั้งสองก้อนยังมี `USED` เป็น `0B` เพราะหน่วยความจำยังเหลือเพื่อ

ค่า `PRIO` เป็นจุดที่ควรอธิบายเพิ่ม เมื่อไม่ได้ระบุ priority เอง kernel จะไล่ค่าติดลบลงเรื่อย ๆ ตามลำดับการเปิดใช้งาน swap ก้อนแรกจึงได้ `-2` และก้อนที่เปิดทีหลังได้ `-3` kernel จะใช้ swap ที่มี priority สูงกว่าก่อนเสมอ ในที่นี้คือใช้ partition จนเต็มแล้วจึงลงมาใช้ swap file หากต้องการให้ใช้พร้อมกันแบบกระจายโหลด ต้องกำหนด priority ให้เท่ากันด้วย `swapon -p` หรือใส่ option `pri=` ใน `/etc/fstab`

1.5.5 ขั้นที่ 5: ยืนยันว่า swap เพิ่มขึ้นจริง



```
poomipat@DESKTOP-39EEJRM: ~$ free -h; free -h --si
total        used        free      shared  buff/cache   available
Mem:    15Gi       613Mi       14Gi       3.6Mi       618Mi       14Gi
Swap:    6.0Gi         0B       6.0Gi             0          0
total        used        free      shared  buff/cache   available
Mem:    16G        642M        15G       3.7M        648M        15G
Swap:    6.4G         0B        6.4G             0          0
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 5: swap total เพิ่มจาก 4.0Gi เป็น 6.0Gi และการเทียบหน่วยฐาน 1024 กับฐาน 1000

บรรทัด swap แสดง total เท่ากับ 6.0Gi ซึ่งคือ 4 GiB ของ partition เดิมบวกกับ 2 GiB ของ swap file ที่เพิ่งเพิ่มเข้าไป เท่ากับว่าการเพิ่ม swap สำเร็จตามเป้าหมาย

คำสั่งที่สองในภาพคือ `free -h --si` ซึ่งให้ผลเป็น 6.4G และ 16G เพราะ `--si` เปลี่ยนไปใช้หน่วยฐาน 1000 (GB) แทนฐาน 1024 (GiB) ตัวเลขที่ต่างกันนี้ไม่ได้แปลว่าหน่วยความจำเปลี่ยนแปลง เป็นเพียงการเลือกฐานของหน่วยที่ต่างกันเท่านั้น

1.5.6 ขั้นที่ 6: ทำให้ถาวร และข้อจำกัดที่พบบน WSL2

บน Linux ทั่วไป การทำให้ swap file กลับมาเองหลัง reboot ทำได้ด้วยการเพิ่มบรรทัดต่อไปนี้ลงใน `/etc/fstab`

```
/swapfile none swap sw 0 0
```

Code 3: รูปแบบของบรรทัด swap ใน `/etc/fstab`

ทั้งหกช่องมีความหมายตามลำดับคือ อุปกรณ์หรือ file ที่จะใช้, จุด mount (swap ไม่มีจุด mount จึงใส่ none), ชนิดของ file system (swap), option (sw คือค่าเริ่มต้นสำหรับ swap), ค่าสำหรับ dump และลำดับการตรวจ file system ด้วย `fsck` ซึ่ง swap ไม่ต้องตรวจจึงเป็น 0 ทั้งสองช่อง

```
poomipat@DESKTOP-39EEJRM: ~$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab; sudo systemctl daemon-reload; systemctl --no-pager list-units --type=swap
/swapfile none swap sw 0 0
UNIT LOAD ACTIVE SUB DESCRIPTION
0 loaded units listed. Pass --all to see loaded but inactive units, too.
To show all installed unit files use 'systemctl list-unit-files'.
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 6: บรรทัดเข้า `/etc/fstab` สำเร็จ แต่ systemd ตอบว่าไม่มี swap unit ที่ load อยู่เลย

ผลที่ได้กลับไม่เป็นไปตามตำราคือ หลังสั่ง `systemctl daemon-reload` แล้วตรวจด้วย `systemctl list-units --type=swap` ระบบตอบว่า 0 loaded units listed แสดงว่า systemd ไม่ได้สร้าง unit ชื่อ `swapfile.swap` ให้ตามที่ควรจะเป็น เมื่อไล่หาสาเหตุจาก journal ของระบบพบข้อความที่ชี้สาเหตุโดยตรง

```
poomipat@DESKTOP-39EEJRM: ~$ systemd-detect-virt --container; journalctl -b --no-pager | grep -i "ignoring swap entry"; sudo swapoff /swapfile; sudo swapon --show; sudo swapon -a; swapon --show
Sep 20 22:07:39 DESKTOP-39EEJRM systemd-fstab-generator[1652]: Running in a container, ignoring swap entry for /swapfile.
NAME TYPE SIZE USED PRIO
/dev/sdc partition 4G 0B -2
NAME TYPE SIZE USED PRIO
/dev/sdc partition 4G 0B -2
/swapfile file 2G 0B -3
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 7: สาเหตุที่แท้จริง: systemd มองว่า WSL2 เป็น container จึงข้าม swap entry ใน `/etc/fstab`

ภาพนี้อธิบายเรื่องทั้งหมดได้ในภาพเดียว เริ่มจาก `systemd-detect-virt --container` ตอบว่า `ws1` คือ systemd รู้ตัวว่ากำลังทำงานอยู่ภายใน container จากนั้นบรรทัดที่ดึงมาจาก journal ระบุว่า `systemd-fstab-generator: Running in a container, ignoring swap entry for /swapfile`. กล่าวคือ ตัว generator ที่ทำหน้าที่แปลง `/etc/fstab` เป็น systemd unit จะข้ามบรรทัด swap ทุกบรรทัดที่เมื่อทำงานอยู่ใน container เพราะตามหลักการแล้ว container ไม่ควรจัดการ swap ของเครื่องเอง

เพื่อพิสูจน์ว่าบรรทัดใน `/etc/fstab` นั้นถูกต้อง ภาพเดียวกันจึงทดสอบต่อด้วยการสั่ง `swapoff` ให้เหลือ swap ก้อนเดียวก่อน แล้วสั่ง `sudo swapon -a` ซึ่งเป็นคำสั่งที่อ่าน `/etc/fstab` โดยตรงไม่ผ่าน systemd ผลคือ `/swapfile` กลับมาทำงานอีกครั้งพร้อม priority -3 เหมือนเดิม สรุปได้ว่าปัญหาไม่ได้อยู่ที่บรรทัดใน `/etc/fstab` แต่อยู่ที่ systemd บน WSL2 ต่างหากที่เลือกจะไม่อ่านมัน

วิธีทำให้ swap ถาวรบน WSL2 มีสองทางที่ใช้ได้จริง

1. สั่ง `swapon -a` ตอน WSL2 เริ่มทำงาน โดยเพิ่มค่าต่อไปนี้ใน `/etc/ws1.conf` ฝั่ง Linux

```
[boot]
command = swapon -a
```

2. ปรับขนาด swap ที่ WSL2 สร้างให้เองแทนการเพิ่ม swap file ใหม่ โดยสร้าง file c:\Users\<user>\.wslconfig ฝั่ง Windows แล้วกำหนด

```
[wsl2]
swap = 8GB
```

จากนั้นสั่ง `wsl --shutdown` ให้ virtual machine ปิดแล้วเปิดใหม่ ขนาดของ partition `/dev/sdc` จะเปลี่ยนตามค่าที่ตั้งไว้ วิธีนี้เป็นวิธีที่ตรงกับสถาปัตยกรรมของ WSL2 มากที่สุด เพราะ swap ของ WSL2 ถูกจัดการจากฝั่ง Windows ไม่ใช่จากฝั่ง Linux

1.5.7 ขั้นที่ 7: ลด swap space

```
poomipat@DESKTOP-39EEJRM: ~$ sudo swapon --show; free -h
NAME      TYPE      SIZE USED PRIO
/dev/sdc partition 4G   0B   -2
Mem:       15Gi   613Mi   14Gi   3.6Mi   178Mi   14Gi
Swap:      4.0Gi   0B     4.0Gi
```

Figure 8: `swapon` ปิดการใช้งาน swap file ทำให้ swap รวมลดจาก 6.0Gi กลับเป็น 4.0Gi

การลด swap ทำได้ด้วยคำสั่ง `sudo swapon /swapfile` ซึ่งจะย้าย page ทั้งหมดที่ค้างอยู่ใน swap area ก่อนนั้นกลับเข้าหน่วยความจำหลักหรือย้ายไป swap ก่อนอื่นก่อน แล้วจึงถอด swap area ออกจากระบบ ข้อควรระวังคือ **ต้องมีหน่วยความจำว่างพอจะรับ page เหล่านั้นกลับ** มิฉะนั้นคำสั่งจะล้มเหลวด้วยข้อความ `Cannot allocate memory` ในการทดลองนี้ swap ยังไม่ถูกใช้งานเลย คำสั่งจึงจบทันที ผลลัพธ์คือเหลือ swap เพียง `/dev/sdc` และ Swap total กลับเป็น 4.0Gi

อีกจุดที่สังเกตได้คือ `buff/cache` ลดจาก 618 MiB เหลือ 178 MiB เพราะ page cache ที่ระบบใช้กับ file `/swapfile` ถูกปล่อยคืนไปพร้อมกัน

1.5.8 ขั้นที่ 8: ลบ file และคืนค่าระบบให้เหมือนเดิม

```
poomipat@DESKTOP-39EEJRM: ~$ sudo sed -i 's|"/swapfile|d"|' /etc/fstab; sudo rm -f /swapfile; cat /etc/fstab; ls -lh /swapfile; free -h
# UNCONFIGURED FSTAB FOR BASE SYSTEM
ls: cannot access '/swapfile': No such file or directory
total      used      free      shared  buff/cache  available
Mem:       15Gi   611Mi   14Gi   3.6Mi   179Mi   14Gi
Swap:      4.0Gi   0B     4.0Gi
```

Figure 9: ลบบรรทัดใน `/etc/fstab` และลบ file ที่ระบบกลับสู่สภาพก่อนการทดลอง

เมื่อ swap file ถูกปิดการใช้งานแล้วจึงลบ file ที่ได้อย่างปลอดภัยด้วย `sudo rm /swapfile` พร้อมกับลบบรรทัดที่เพิ่มไว้ใน `/etc/fstab` ออก ภาพแสดงว่า `/etc/fstab` เหลือเพียงบรรทัดคอมเมนต์เดิม และ `ls` รายงาน `No such file or directory` ซึ่งยืนยันว่า file ถูกลบจริง ส่วน swap total คือ 4.0Gi เท่ากับตอนเริ่มต้นการทดลองทุกประการ

ลำดับที่ห้ามสลับ: ต้องสั่ง `swapon` ก่อนลบ file เสมอ หากลบ file ที่ขณะที่ swap ยังเปิดใช้งานอยู่ kernel จะยังอ้างถึงพื้นที่นั้นต่อไปจนกว่าจะ reboot และอาจทำให้ระบบไม่เสถียร

1.5.9 การปรับพฤติกรรมการใช้ swap ด้วย `vm.swappiness`

การเพิ่มหรือลดขนาด swap เป็นการเปลี่ยน *ปริมาณ* ที่มีให้ใช้ ส่วนการปรับ `vm.swappiness` เป็นการเปลี่ยน *ความเต็มใจ* ของ kernel ที่จะใช้ swap ซึ่งมักต้องทำควบคู่กัน

```
poomipat@DESKTOP-39EEJRM:~$ cat /proc/sys/vm/swappiness; sudo sysctl vm.swappiness=10; cat /proc/sys/vm/swappiness; sudo sysctl vm.swappiness=60; cat /proc/sys/vm/swappiness
60
vm.swappiness = 10
10
vm.swappiness = 60
60
poomipat@DESKTOP-39EEJRM:~$
```

Figure 10: ปรับ vm.swappiness จาก 60 เป็น 10 แล้วตั้งกลับเป็น 60

ค่าเริ่มต้นของระบบคือ 60 เมื่อสั่ง `sudo sysctl vm.swappiness=10` ค่าใน `/proc/sys/vm/swappiness` เปลี่ยนเป็น 10 ทันทีโดยไม่ต้อง reboot ค่าที่ต่ำลงหมายความว่า kernel จะพยายามเรียกคืน page cache ก่อนและหลีกเลี่ยงการย้าย anonymous page ลง swap ส่วนค่าที่สูงขึ้นหมายถึงยอมใช้ swap ได้ง่ายขึ้น การทดลองจบลงด้วยการตั้งค่ากลับเป็น 60 เพื่อคืนสภาพระบบ

ค่าที่ตั้งด้วย `sysctl` จะหายไปเมื่อ reboot หากต้องการให้ถาวรต้องเพิ่มบรรทัด `vm.swappiness=10` ลงใน `/etc/sysctl.conf` หรือสร้าง file ใหม่ใน `/etc/sysctl.d/`

1.5.10 สรุปคำสั่งทั้งหมดของข้อ 1.2

Table 3: สรุปคำสั่งสำหรับเพิ่มและลด swap space บน Linux

เป้าหมาย	คำสั่ง	หน้าที่
เพิ่ม swap	<code>sudo falldate -l 2G /swapfile</code> <code>sudo chmod 600 /swapfile</code> <code>sudo mkswap /swapfile</code> <code>sudo swapon /swapfile</code> <code>echo ... sudo tee -a /etc/fstab</code>	จองพื้นที่บน disk ขนาด 2 GiB ปิดสิทธิ์ให้เหลือเฉพาะ root เขียน header ของ swap area และกำหนด UUID สั่งให้ kernel เริ่มใช้งาน swap area นี้ ทำให้กลับมาเองหลัง reboot (ดูข้อจำกัดบน WSL2)
ลบ swap	<code>sudo swapoff /swapfile</code> <code>sudo rm /swapfile</code> แก้ <code>/etc/fstab</code>	ย้าย page กลับเข้า RAM แล้วถอด swap area ออก ลบ file ที่หลังปิดการใช้งานแล้วเท่านั้น ลบบรรทัดที่เพิ่มไว้
ตรวจสอบ	<code>free -h, swapon --show, cat /proc/swaps</code>	ดูขนาดและสถานะการใช้งานของ swap
ปรับพฤติกรรม	<code>sudo sysctl vm.swappiness=10</code>	เปลี่ยนแนวโน้มที่ kernel จะใช้ swap

2 ข้อ 2: การเฝ้าดู Virtual Memory บน Linux

ข้อนี้ใช้เครื่องมือสองตัวจาก package `procps-ng` คือ `free` ซึ่งรายงาน *สถานะ ณ ขณะนั้น* ของหน่วยความจำ และ `vmstat` ซึ่งรายงาน *อัตราการเคลื่อนไหว* ของระบบ ทั้งสองคำสั่งอ่านข้อมูลมาจาก `/proc` ที่ kernel เปิดให้เหมือนกัน แต่จัดรูปแบบและตีความต่างกัน

2.1 การรัน free และผลลัพธ์ที่ได้

```
poomipat@DESKTOP-39EEJRM:~$ free
total        used        free        shared    buff/cache   available
Mem:      16093844    625124    15468720        3652        192696        15468720
Swap:      4194304         0         4194304
```

Figure 11: free แบบไม่ใส่ argument ใด ๆ ค่าทั้งหมดมีหน่วยเป็น KiB

เมื่อรัน `free` เปล่า ๆ ผลลัพธ์เป็นตัวเลขดิบที่มีหน่วยเป็น KiB (1024 bytes) ซึ่งอ่านยากเมื่อค่ามีหลักมาก จึงนิยมเติม `-h` เพื่อให้แปลงหน่วยให้อัตโนมัติ

```
poomipat@DESKTOP-39EEJRM:~$ free -h
              total        used          free      shared  buff/cache   available
Mem:           15Gi        610Mi          14Gi         3.6Mi        188Mi        14Gi
Swap:           4.0Gi           0B          4.0Gi
poomipat@DESKTOP-39EEJRM:~$
```

Figure 12: `free -h` แสดงข้อมูลชุดเดียวกันในหน่วยที่อ่านง่าย

2.1.1 ความหมายของทุกฟิลด์ที่คำสั่งนี้แสดง

ผลลัพธ์ประกอบด้วยสองแถวคือ `Mem:` สำหรับหน่วยความจำหลัก และ `Swap:` สำหรับพื้นที่ swap โดยมีคอลัมน์ดังนี้

Table 4: ความหมายของทุกคอลัมน์ในผลลัพธ์ของคำสั่ง `free`

คอลัมน์	ความหมาย
total	ขนาดทั้งหมดที่ระบบปฏิบัติการมองเห็นและใช้งานได้ อ่านมาจาก <code>MemTotal</code> และ <code>SwapTotal</code> ใน <code>/proc/meminfo</code> คำนี้น้อยกว่า RAM ที่ติดตั้งจริงเล็กน้อยเพราะ firmware และ kernel กันบางส่วนไว้ตั้งแต่ตอนบูต
used	หน่วยความจำที่ถูกใช้งานจริงโดย process ต่าง ๆ คำนวณจาก <code>total</code> ลบด้วย <code>free</code> และ <code>buff/cache</code> จึงเป็นตัวเลขที่บอกภาระจริงของระบบ
free	หน่วยความจำที่ยังไม่ถูกแตะต้องเลย ไม่ได้แปลว่าเป็นหน่วยความจำที่เหลือให้ใช้ทั้งหมด เพราะส่วนที่เป็น cache ก็ขอคืนได้เช่นกัน
shared	หน่วยความจำที่ถูกแบ่งใช้ร่วมกันระหว่าง process ส่วนใหญ่คือ <code>tmpfs</code> และ shared memory segment
buff/cache	ผลรวมของ buffer (ข้อมูล metadata ของ block device) กับ page cache (เนื้อหาของ file ที่เคยอ่านเขียน) หน่วยความจำส่วนนี้ kernel ยืมไปใช้เร่งความเร็ว และคืนให้ process ได้ทันทีที่ต้องการ
available	ค่าประมาณของหน่วยความจำที่ process ใหม่ขอใช้ได้จริงโดยระบบไม่ต้องเริ่ม swap เท่ากับ <code>free</code> บวกกับส่วนของ <code>buff/cache</code> ที่เรียกคืนได้ คำนี้อาจต่ำกว่าที่ควรดูเวลาประเมินว่าเครื่องยังมีหน่วยความจำเหลือหรือไม่

สำหรับแถว `Swap:` มีเพียงสามคอลัมน์แรกที่มีความหมาย คือขนาดรวม ส่วนที่ถูกใช้ไป และส่วนที่ยังว่าง ส่วนคอลัมน์ที่เหลือเว้นว่างไว้เพราะไม่มีแนวคิดเรื่อง cache หรือ available สำหรับ swap

ข้อสังเกตเพิ่มเติม `free` รุ่นเก่า (procps ก่อนรุ่น 3.3.10) เคยมีบรรทัด `-/+ buffers/cache` เพิ่มมาอีกหนึ่งบรรทัดเพื่อบอกว่าถ้าไม่นับ cache แล้วจะเหลือหน่วยความจำเท่าไร ปัจจุบันบรรทัดนั้นถูกตัดออกและแทนที่ด้วยคอลัมน์ `available` ซึ่งแม่นยำกว่า เพราะ kernel เป็นผู้คำนวณให้เองว่าส่วนใดของ cache เรียกคืนได้จริง

2.1.2 สรุปตัวเลขทั้งหมดที่คำสั่ง `free` รายงาน

จากภาพผลการรัน ระบบรายงานหน่วยความจำหลักมีทั้งหมด 16,093,844 KiB ซึ่งเท่ากับ 15 GiB เมื่อแสดงด้วย `-h` ในจำนวนนี้ถูกใช้งานจริงโดย process ต่าง ๆ 625,124 KiB หรือประมาณ 610 MiB ยังไม่ถูกแตะต้องเลย 15,483,812 KiB หรือประมาณ 14 GiB เป็นหน่วยความจำที่แบ่งใช้ร่วมกัน 3,652 KiB หรือ 3.6 MiB และถูก kernel ยืมไปทำ buffer กับ page cache อีก 192,696 KiB หรือ 188 MiB ส่วนค่า `available` ซึ่งเป็นตัวเลขที่สำคัญที่สุดในการประเมินสภาพเครื่องอยู่ที่ 15,468,720 KiB หรือประมาณ 14 GiB กล่าวคือระบบยังมีหน่วยความจำให้ใช้อีกราว 96% ของทั้งหมด สำหรับ swap มีขนาดรวม 4,194,304 KiB หรือ 4.0 GiB ถูกใช้ไป 0 KiB และว่างทั้งหมด 4,194,304 KiB การที่ `used` ของ swap เป็นศูนย์ในขณะที่หน่วยความจำหลักเหลือถึง 14 GiB เป็นภาพของระบบที่ยังไม่มีแรงกดดันด้านหน่วยความจำเลยแม้แต่น้อย kernel จึงไม่จำเป็นต้องย้าย page ไตลง swap

2.2 argument ที่น่าสนใจของ free

```
poomipat@DESKTOP-39EEJRM:~$ free --help
Usage:
free [options]

Options:
-b, --bytes          show output in bytes
--kilo              show output in kilobytes
--mega              show output in megabytes
--giga              show output in gigabytes
--tera              show output in terabytes
--peta              show output in petabytes
-k, --kibi           show output in kibibytes
-m, --mebi           show output in mebibytes
-g, --gibi           show output in gibibytes
--tebi             show output in tebibytes
--pebi             show output in pebibytes
-h, --human          show human-readable output
--si               use powers of 1000 not 1024
-l, --lohi           show detailed low and high memory statistics
-L, --line           show output on a single line
-t, --total          show total for RAM + swap
-s, --committed     show committed memory and commit limit
-v, --seconds N      repeat printing every N seconds
-c N, --count N      repeat printing N times, then exit
-w, --wide           wide output

--help             display this help and exit
-V, --version       output version information and exit

For more details see free(1).
poomipat@DESKTOP-39EEJRM:~$
```

Figure 13: free --help แสดง option ทั้งหมดที่มีให้เลือกใช้

จากรายการทั้งหมด รายงานนี้เลือกมาสองชุดที่ใช้งานได้จริงในสถานการณ์ต่างกัน

2.2.1 argument ที่ 1: -w ร่วมกับ -t

หน้าที่ -w (wide) สั่งให้แยกคอลัมน์ buff/cache ที่ปกติรวมกันออกเป็นสองคอลัมน์คือ buffers กับ cache ส่วน -t (total) เพิ่มแถว Total: ที่รวมหน่วยความจำหลักกับ swap เข้าด้วยกัน การใช้สองตัวนี้พร้อมกันทำให้เห็นทั้งรายละเอียดที่ละเอียดขึ้นและภาพรวมในคำสั่งเดียว

```
free -h
free -h -w -t
```

Code 4: เปรียบเทียบผลของ free แบบปกติกับแบบ -w -t

```
poomipat@DESKTOP-39EEJRM:~$ free -h; free -h -w -t
Mem:               total      used      free      shared  buff/cache   available
Swap:             4.0Gi       0B       4.0Gi       3.6Mi      188Mi       14Gi

Mem:               total      used      free      shared  buffers      cache   available
Swap:             4.0Gi       0B       4.0Gi       3.6Mi      4.7Mi      183Mi      14Gi
Total:            19Gi       610Mi      18Gi

poomipat@DESKTOP-39EEJRM:~$
```

Figure 14: -w แยก buffers ออกจาก cache และ -t เพิ่มแถวผลรวม

อธิบายผลลัพธ์ สองแถวบนคือผลของ free -h ตามปกติ ซึ่งรายงาน buff/cache รวมกันเป็น 188 MiB ส่วนสามแถวล่างคือผลของ free -h -w -t ที่แยกให้เห็นว่าในจำนวนนั้นเป็น buffers เพียง 4.7 MiB และเป็น cache ถึง 183 MiB ซึ่งบอกได้ว่าหน่วยความจำที่ kernel ยืมไปเกือบทั้งหมดเป็น page cache ของ file ไม่ใช่ metadata ของ block device แถวสุดท้าย Total: แสดงผลรวมของหน่วยความจำหลักกับ swap คือ 19 GiB โดยใช้ไป 610 MiB และเหลือ 18 GiB ซึ่งเป็นมุมมองที่มีประโยชน์เมื่อต้องการรู้ว่าระบบมีพื้นที่เก็บ page ได้ทั้งหมดเท่าไรเมื่อนับ swap ด้วย

2.2.2 argument ที่ 2: -s ร่วมกับ -c

หน้าที่ -s N สั่งให้พิมพ์ผลซ้ำทุก N วินาที และ -c N กำหนดจำนวนรอบแล้วหยุดเอง เมื่อใช้คู่กันจะได้เครื่องมือเฝ้าดูหน่วยความจำแบบสั้น ๆ โดยไม่ต้องเปิด top หรือกด Ctrl+C เอง เหมาะกับการดูว่าหน่วยความจำเปลี่ยนแปลงอย่างไรระหว่างที่ program กำลังทำงาน

```
poomipat@DESKTOP-39EEJRM:~$ free -h -s 2 -c 3
total        used        free      shared  buff/cache   available
Mem:          15Gi        609Mi       14Gi         3.6Mi        188Mi        14Gi
Swap:          4.0Gi           0B         4.0Gi
total        used        free      shared  buff/cache   available
Mem:          15Gi        609Mi       14Gi         3.6Mi        188Mi        14Gi
Swap:          4.0Gi           0B         4.0Gi
total        used        free      shared  buff/cache   available
Mem:          15Gi        610Mi       14Gi         3.6Mi        188Mi        14Gi
Swap:          4.0Gi           0B         4.0Gi
poomipat@DESKTOP-39EEJRM:~$
```

Figure 15: free -h -s 2 -c 3 พิมพ์ผลสามรอบ ห่างกันรอบละ 2 วินาที แล้วจบการทำงาน

อธิบายผลลัพธ์ ตารางถูกพิมพ์ซ้ำสามครั้งตามที่กำหนดด้วย -c 3 โดยเว้นระยะครั้งละ 2 วินาทีตาม -s 2 แล้วคำสั่งจบเอง เมื่อเทียบตัวเลขระหว่างรอบจะเห็นว่า used ขยับจาก 609 MiB เป็น 609 MiB และ 610 MiB ตามลำดับ ส่วนค่าอื่นแทบไม่เปลี่ยน ซึ่งสะท้อนว่าเครื่องอยู่ในสภาวะว่างงาน หากนำคำสั่งนี้ไปรันระหว่างที่ program ใช้หน่วยความจำหนัก จะเห็น used ได้ขึ้นและ available ลดลงอย่างชัดเจนในแต่ละรอบ

2.3 การติดตั้งและรัน vmstat

```
poomipat@DESKTOP-39EEJRM:~$ which vmstat; dpkg -l procps | tail -3; vmstat --version
/usr/bin/vmstat
||/ Name          Version          Architecture Description
+++=====
ii procps         2:4.0.4-4ubuntu3.2 amd64          /proc file system utilities
vmstat from procps-ng 4.0.4
poomipat@DESKTOP-39EEJRM:~$
```

Figure 16: ตรวจสอบว่ามี vmstat อยู่แล้ว โดยมาจาก package procps

ใบงานระบุว่าหากไม่มี vmstat ให้ติดตั้งก่อน จึงตรวจสอบด้วย which vmstat ซึ่งตอบว่าอยู่ที่ /usr/bin/vmstat และ dpkg -l procps ยืนยันว่า package procps version 2:4.0.4-4ubuntu3.2 ถูกติดตั้งสมบูรณ์แล้ว (สถานะ ii) ส่วนตัวโปรแกรมรายงานตัวเองว่าเป็น vmstat from procps-ng 4.0.4 หากเป็นเครื่องที่ยังไม่มี ติดตั้งได้ด้วยคำสั่ง

```
sudo apt update
sudo apt install -y procps
```

Code 5: คำสั่งติดตั้ง vmstat บน Ubuntu หรือ Debian

```
poomipat@DESKTOP-39EEJRM:~$ vmstat
procs -----memory----- --swap--  -----io----- -system-- -----cpu-----
r  b  swpd  free  buff  cache  si  so  bi  bo  in  cs  us  sy  id  wa  st  gu
1  0      0 15071048 8524 591736 0  0 419 271 206  0  0  0 100  0  0  0
poomipat@DESKTOP-39EEJRM:~$
```

Figure 17: vmstat แบบไม่ใส่ argument ให้ข้อมูลหนึ่งแถวซึ่งเป็นค่าเฉลี่ยตั้งแต่บูตเครื่อง

2.3.1 ความหมายของทุกคอลัมน์ที่คำสั่งนี้แสดง

Table 5: ความหมายของทุกคอลัมน์ในผลลัพธ์ของคำสั่ง `vmstat`

กลุ่ม	คอลัมน์	ความหมาย
procs	r	จำนวน process ที่พร้อมทำงานและกำลังรอคิว CPU ถ้าค่านี้สูงกว่าจำนวน core เป็นเวลานานแปลว่า CPU เป็นคอขวด
	b	จำนวน process ที่ติดอยู่ในสถานะ uninterruptible sleep ซึ่งเกือบทั้งหมดคือการรอ disk
memory	swpd	ปริมาณหน่วยความจำที่ถูกย้ายไปอยู่บน swap แล้ว
	free	หน่วยความจำที่ยังว่างอยู่
	buff	หน่วยความจำที่ใช้เป็น buffer สำหรับ metadata ของ block device
	cache	หน่วยความจำที่ใช้เป็น page cache ของ file
swap	si	อัตราการอ่าน page กลับจาก swap เข้าหน่วยความจำ (swap in) หน่วยเป็น KiB ต่อวินาที
	so	อัตราการเขียน page จากหน่วยความจำลง swap (swap out) ถ้าสองค่านี้สูงต่อเนื่องคืออาการ thrashing
io	bi	จำนวน block ที่อ่านเข้ามาจาก block device ต่อวินาที
	bo	จำนวน block ที่เขียนออกไปยัง block device ต่อวินาที
system	in	จำนวน interrupt ต่อวินาที รวม interrupt ของ timer ด้วย
	cs	จำนวนครั้งของการสลับ context ต่อวินาที
cpu	us	เปอร์เซ็นต์เวลาที่ CPU ใช้กับ code ใน user space
	sy	เปอร์เซ็นต์เวลาที่ CPU ใช้กับ code ของ kernel
	id	เปอร์เซ็นต์เวลาที่ CPU ว่าง
	wa	เปอร์เซ็นต์เวลาที่ CPU ว่างเพราะติดรอ I/O ค่านี้สูงแปลว่า disk เป็นคอขวด
	st	เปอร์เซ็นต์เวลาที่ถูก hypervisor ยืมไปให้ virtual machine อื่น
	gu	เปอร์เซ็นต์เวลาที่ใช้รัน guest virtual machine (มีใน procps-ng รุ่นใหม่)

ข้อควรระวังที่สำคัญที่สุดของ `vmstat` แถวแรกที่พิมพ์ออกมาเสมอ **ไม่ใช่ค่าขณะนั้น** แต่เป็นค่าเฉลี่ยสะสมตั้งแต่บูตเครื่อง หากต้องการค่าที่เป็นปัจจุบันจริงต้องสั่งให้เก็บตัวอย่างเป็นช่วง เช่น `vmstat 1 5` แล้วดูตั้งแต่แถวที่สองเป็นต้นไป

2.3.2 สรุปตัวเลขทั้งหมดที่คำสั่ง `vmstat` รายงาน

จากภาพผลการรัน กลุ่ม procs รายงานว่ามี process ที่พร้อมทำงาน 1 ตัว ซึ่งก็คือตัว `vmstat` เอง และไม่มี process ใดติดรอ disk เลย (b เท่ากับ 0) กลุ่ม memory แสดงว่ายังไม่มีข้อมูลใดถูกย้ายลง swap (swpd เท่ากับ 0) มีหน่วยความจำว่าง 15,071,048 KiB ใช้เป็น buffer 8,524 KiB และใช้เป็น page cache 591,736 KiB กลุ่ม swap เป็นศูนย์ทั้ง si และ so ยืนยันอีกครั้งว่าไม่มีการสลับ page กับ disk เกิดขึ้นเลยนับตั้งแต่บูต กลุ่ม io รายงานค่าเฉลี่ยการอ่าน 419 block ต่อวินาทีและการเขียน 271 block ต่อวินาที ซึ่งเป็นภาระ I/O ของช่วงที่ระบบบูตและเปิด program เป็นหลัก กลุ่ม system รายงาน interrupt เฉลี่ย 206 ครั้งต่อวินาที ส่วนกลุ่ม cpu ชี้ว่าเครื่องแทบไม่ได้ทำงานเลย คือ us, sy, wa, st และ gu เป็นศูนย์ทั้งหมด ขณะที่ id เท่ากับ 100 เปอร์เซ็นต์ ภาพรวมทั้งหมดคือระบบที่มีหน่วยความจำเหลือเฟือ ไม่มีแรงกดดันให้ต้องใช้ swap และ CPU อยู่ในสภาวะว่าง

สิ่งที่สังเกตพบระหว่างการทดลอง คอลัมน์ cs ในแถวแรกรายงานค่าเป็น 0 ทั้งที่ระบบต้องมีการสลับ context เกิดขึ้นแน่นอน เมื่อตรวจสอบเพิ่มเติมพบว่าคำสั่ง `vmstat -s` รายงานตัวนับสะสมว่ามีการสลับ context ไปแล้ว 261,886 ครั้ง และเมื่อสั่ง `vmstat -s m 1 5` เพื่อเก็บตัวอย่างเป็นช่วง แถวที่สองเป็นต้นไปก็รายงาน cs เป็น 100, 26, 90 และ 51 ตามปกติ สรุปได้ว่าค่า 0 นี้เกิดขึ้นเฉพาะกับแถวค่าเฉลี่ยตั้งแต่บูตบนสภาพแวดล้อม WSL2 เท่านั้น ส่วนคอลัมน์

in ในแถวเดียวกันกลับคำนวณได้ถูกต้อง (206 ครั้งต่อวินาที ซึ่งสอดคล้องกับตัวนับ interrupts ที่ 367,047 ครั้ง) ดังนั้นเมื่อต้องอ่านค่า cs จริง ควรใช้ค่าจากแถวที่เก็บตัวอย่างเป็นช่วงเสมอ

2.4 argument ที่น่าสนใจของ vmstat

2.4.1 argument ที่ 1: -s M ร่วมกับารระบุ interval และ count

หน้าที่ -s ใช้เปลี่ยนหน่วยของคอลัมน์ในกลุ่ม memory และ swap โดย M หมายถึงเมกะไบต์ (ยังเลือก k, K, m ได้ด้วย) ส่วนตัวเลขสองตัวที่ตามหลังคำสั่งคือ interval และ count ตามลำดับ เมื่อรวมกันจึงได้คำสั่งที่เก็บตัวอย่างสภาพระบบเป็นช่วงเวลาในหน่วยที่อ่านง่าย

```
poomipat@DESKTOP-39EEJRM:~$ vmstat -S M 1 5
procs -----memory----- --swap-- --io-- --system-- -----cpu-----
r  b   swpd   free   buff   cache   si   so   bi   bo   in   cs   us   sy   id   wa   st   gu
1  0     0   15170    1    132    0    0   369  239  190    0    0    0  100    0    0    0
0  0     0   15170    1    132    0    0    0    0   101  100    0    0  100    0    0    0
1  0     0   15170    1    132    0    0    0    0   36   26    0    0  100    0    0    0
0  0     0   15170    1    132    0    0    0    0   72   90    0    0  100    0    0    0
0  0     0   15170    1    132    0    0    0    4   52   51    0    0  100    0    0    0
poomipat@DESKTOP-39EEJRM:~$
```

Figure 18: vmstat -S M 1 5 เก็บตัวอย่างทุก 1 วินาที จำนวน 5 รอบ ในหน่วย MB

อธิบายผลลัพธ์ ตัวเลขในกลุ่ม memory เปลี่ยนจากหลักล้าน KiB มาเป็น 15,170 MB สำหรับ free, 1 MB สำหรับ buff และ 132 MB สำหรับ cache ซึ่งอ่านเทียบกับขนาดหน่วยความจำจริงได้ง่ายกว่ามาก แถวแรกยังคงเป็นค่าเฉลี่ยตั้งแต่บูต (bi 369, bo 239, in 190) ส่วนแถวที่สองถึงแถวที่ห้าคือค่าจริงของแต่ละวินาที ซึ่งเห็นชัดว่า bi และ bo กลายเป็น 0 เกือบทั้งหมดเพราะไม่มีการอ่านเขียน disk จริงในช่วงนั้น ขณะที่ in และ cs แกว่งอยู่ในช่วง 36–101 และ 26–100 ตามจังหวะการทำงานเบื้องหลังของระบบ นี่คือรูปแบบการใช้งานที่ถูกต้องของ vmstat คือดูแนวโน้มจากหลายแถว ไม่ใช่ตัดสินจากแถวเดียว

2.4.2 argument ที่ 2: -s ร่วมกับ -s M

หน้าที่ -s (stats) เปลี่ยนรูปแบบการแสดงผลจากตารางเป็นรายการตัวนับสะสมตั้งแต่บูตเครื่องประมาณ 28 บรรทัด ครอบคลุมทั้งข้อมูลหน่วยความจำ ข้อมูล CPU ticks และเหตุการณ์ของ virtual memory ที่ตารางปกติไม่ได้แสดง เหมาะกับการตรวจย้อนหลังว่าตั้งแต่เปิดเครื่องมาระบบเคยเจอแรงกดดันด้านหน่วยความจำหรือไม่

```
poomipat@DESKTOP-39EEJRM: ~$ vmstat -s -S M
15716 M total memory
599 M used memory
45 M active memory
112 M inactive memory
15173 M free memory
1 M buffer memory
132 M swap cache
4096 M total swap
0 M used swap
4096 M free swap
2277 non-nice user cpu ticks
1226 nice user cpu ticks
1877 system cpu ticks
4371529 idle cpu ticks
554 IO-wait cpu ticks
0 IRQ cpu ticks
196 softirq cpu ticks
0 stolen cpu ticks
0 non-nice guest cpu ticks
0 nice guest cpu ticks
701163 K paged in
454116 K paged out
0 pages swapped in
0 pages swapped out
367047 interrupts
261886 CPU context switches
1789916206 boot time
3279 forks
poomipat@DESKTOP-39EEJRM: ~$
```

Figure 19: `vmstat -s -S M` แสดงตัวนับสะสมทั้งหมดตั้งแต่บูตเครื่อง

อธิบายผลลัพธ์ กลุ่มแรกเป็นข้อมูลหน่วยความจำ ได้แก่ total memory 15,716 MB, used memory 599 MB, active memory 45 MB, inactive memory 112 MB, free memory 15,173 MB, buffer memory 1 MB และ swap cache 132 MB ตามด้วยข้อมูล swap คือ total swap 4,096 MB โดย used swap เป็น 0 และ free swap เต็มจำนวน

กลุ่มที่สองเป็น CPU ticks สะสม ได้แก่ non-nice user 2,277, nice user 1,226, system 1,877, idle 4,371,529, IO-wait 554, IRQ 0, softirq 196, stolen 0 และ guest 0 ทั้งสองแบบ สัดส่วนของ idle ที่มากกว่าค่าอื่นหลายพันเท่ายืนยันว่าเครื่องอยู่ในสภาวะว่างมาตลอด

กลุ่มสุดท้ายเป็นเหตุการณ์ของ virtual memory ซึ่งเป็นส่วนที่ตรงกับหัวข้อของปฏิบัติการนี้มากที่สุด ได้แก่ paged in 701,163 KiB และ paged out 454,116 KiB ซึ่งคือปริมาณข้อมูลที่ระบบอ่านเข้าและเขียนออกกับ disk ผ่านกลไก paging ตั้งแต่บูต ส่วน pages swapped in และ pages swapped out เป็น 0 **ทั้งคู่** ซึ่งเป็นหลักฐานยืนยันว่าตั้งแต่เปิดเครื่องมาไม่เคยมี page ไถ่ถูกย้ายลง swap เลยแม้แต่นับเดียว ปิดท้ายด้วย interrupts 367,047 ครั้ง, CPU context switches 261,886 ครั้ง, boot time 1,789,916,206 (เวลาที่บูตในรูปแบบ Unix timestamp) และ forks 3,279 ครั้ง

ความแตกต่างระหว่าง paging กับ swapping ที่เห็นได้จากตัวเลขชุดนี้ ตัวเลข paged in และ paged out ที่สูง ในขณะที่ pages swapped in และ out เป็นศูนย์ อธิบายได้ว่าการย้ายข้อมูลเกือบทั้งหมดเป็นการอ่าน file เข้ามาใน page cache และเขียนกลับลง file system ตามปกติ ไม่ใช้การนำ anonymous page ไปพักไว้บน swap เพราะหน่วยความจำยังเหลือเฟือ ซึ่งสอดคล้องกับค่าของ free และ vmstat ทุกภาพก่อนหน้านี้

2.5 ความสอดคล้องระหว่างตัวเลขของ free และ vmstat

เนื่องจากทั้งสองคำสั่งอ่านข้อมูลมาจากแหล่งเดียวกัน ตัวเลขจึงต้องสอดคล้องกันเสมอเมื่อปรับหน่วยให้ตรงกัน ตัวอย่างเช่น free รายงาน total เท่ากับ 16,093,844 KiB เมื่อหารด้วย 1024 จะได้ 15,716 MB ซึ่งตรงกับบรรทัด total memory ของ `vmstat -s -S M` พอดี และ swap total 4,194,304 KiB ก็เท่ากับ 4,096 MB ตามที่ `vmstat -s` รายงานเช่นกัน ความต่างของตัวเลขในกลุ่ม free และ cache ระหว่างภาพแต่ละภาพเกิดจากการรันคนละช่วงเวลาเท่านั้น เพราะ page cache เปลี่ยนแปลงตลอดเวลาตามการอ่านเขียน file ของระบบ

สรุปคือ free เหมาะกับการตอบคำถามว่า ตอนนี้หน่วยความจำเหลือเท่าไร ส่วน vmstat เหมาะกับการตอบคำถามว่า ระบบกำลังทำงานหนักแค่ไหนและมีการสลับ page เกิดขึ้นหรือไม่ การใช้งานจริงจึงมักใช้ทั้งสองคำสั่งควบคู่กัน

3 ข้อ 3: การเฝ้าดู Virtual Memory บน MS Windows

ฝั่ง Windows ใช้เครื่องมือชื่อ Resource Monitor ซึ่งติดมากับระบบปฏิบัติการอยู่แล้ว เครื่องมือนี้แสดงข้อมูลหน่วยความจำได้ละเอียดกว่า Task Manager มาก โดยเฉพาะการแยกให้เห็นว่าหน่วยความจำที่ใช้ไปนั้นอยู่ในสถานะใดของระบบ

3.1 การเปิด Resource Monitor

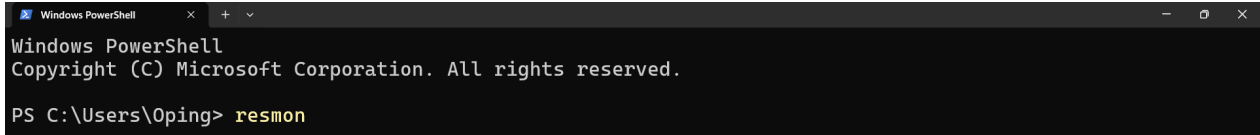


Figure 20: เปิด Resource Monitor ด้วยคำสั่ง `resmon` ใน PowerShell

พิมพ์คำสั่ง `resmon` ใน PowerShell แล้วกด Enter ระบบจะเปิดหน้าต่าง Resource Monitor ขึ้นมา (เปิดผ่าน `perfmon /res` หรือกดปุ่ม Open Resource Monitor ที่แท็บ Performance ของ Task Manager ก็ได้ผลเหมือนกัน) จากนั้นเลือกแท็บ **Memory** ซึ่งเป็นแท็บที่รวมข้อมูลด้าน virtual memory ไว้ทั้งหมด

3.2 ข้อมูลที่แสดงในแท็บ Memory

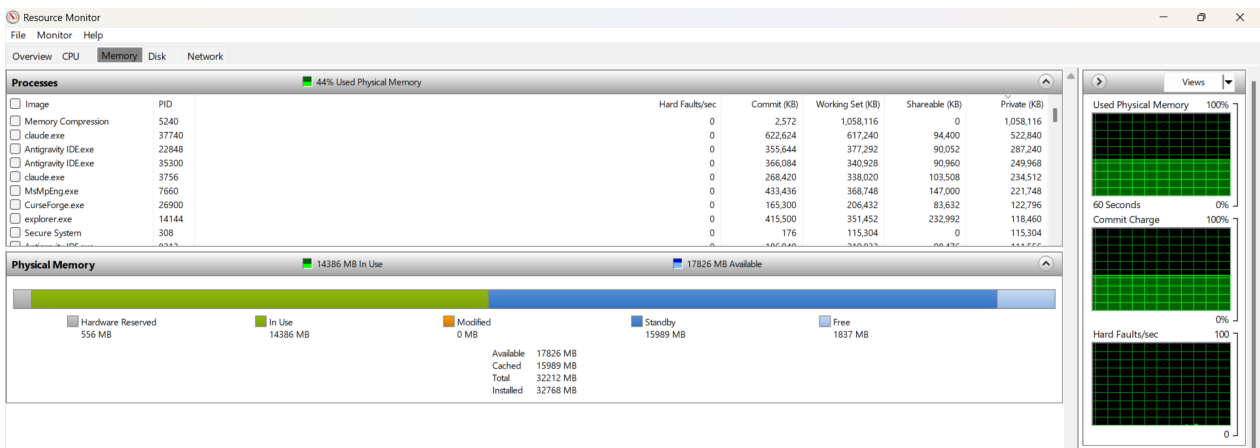


Figure 21: แท็บ Memory ของ Resource Monitor ประกอบด้วยตาราง Processes แถบ Physical Memory และกราฟด้านขวา

หน้าจอนี้แบ่งออกเป็นสามส่วนที่ทำงานเสริมกัน

3.2.1 ส่วนที่ 1: ตาราง Processes

ตารางด้านบนไล่เรียง process ทุกตัวในระบบ เรียงจากตัวที่ใช้หน่วยความจำมากที่สุดลงมา แต่ละคอลัมน์มีความหมายดังนี้

Table 6: ความหมายของคอลัมน์ในตาราง Processes ของแท็บ Memory

คอลัมน์	ความหมาย
Image	ชื่อ file ที่ทำงานเป็น process นั้น
PID	หมายเลขประจำตัวของ process
Hard Faults/sec	จำนวน hard fault ต่อวินาที คือจำนวนครั้งที่ต้องไปอ่านข้อมูลกลับจาก disk เพราะ page ที่ต้องการไม่อยู่ใน RAM ค่านี้คือตัวชี้วัดที่สำคัญที่สุดของหน้าจอนี้ ถ้าสูงต่อเนื่องแปลว่าหน่วยความจำไม่พอจริง
Commit (KB)	ปริมาณ virtual memory ที่ระบบจองโควตาไว้ให้ process นี้ในบัญชี commit charge ไม่ว่า จะอยู่ใน RAM หรือถูกย้ายลง pagefile.sys แล้วก็ตาม
Working Set (KB)	ปริมาณหน่วยความจำที่ process นี้ถือครองอยู่ใน RAM จริง ณ ขณะนั้น เทียบได้กับค่า RSS ของ Linux
Shareable (KB)	ส่วนของ working set ที่แบ่งใช้ร่วมกับ process อื่นได้ เช่น code ของ DLL ที่หลาย program ใช้ร่วมกัน
Private (KB)	ส่วนของ working set ที่เป็นของ process นี้เท่านั้น ใครมาใช้ร่วมไม่ได้ ค่านี้คือหน่วยความจำที่จะได้คืนจริงถ้าปิด process นั้น

ความสัมพันธ์ที่ตรวจสอบได้จากตัวเลขในภาพคือ Working Set เท่ากับ Shareable บวก Private เสมอ เช่น c:\Claude.exe (PID 37740) มี working set 617,240 KB ซึ่งเท่ากับ shareable 94,400 KB บวก private 522,840 KB พอดี

อีกแถวที่น่าสนใจเป็นพิเศษคือ **Memory Compression** (PID 5240) ซึ่งเป็น process ของระบบที่ทำหน้าที่เก็บ page ที่ถูกบีบอัดไว้ใน RAM ตามที่อธิบายไว้ในข้อ 1 จะเห็นว่ามี working set สูงถึง 1,058,116 KB (ราว 1 GB) และเป็น private ทั้งหมด แต่มี Commit เพียง 2,572 KB เพราะหน่วยความจำที่มันถือครองคือข้อมูลของ process อื่นที่ถูกบีบอัดเก็บไว้ ไม่ใช่หน่วยความจำที่ตัวมันเองขอจองมาใช้

3.2.2 ส่วนที่ 2: แถบ Physical Memory

แถบกราฟตรงกลางแบ่งหน่วยความจำจริงทั้งเครื่องออกเป็นห้าส่วนตามสถานะของ page ซึ่งตรงกับรายการ page ระดับระบบที่อธิบายไว้ในหัวข้อ 1.3

3.2.3 ส่วนที่ 3: กราฟด้านขวา

กราฟสามช่องทางขวาแสดงแนวโน้มย้อนหลัง 60 วินาที ได้แก่ **Used Physical Memory** (สัดส่วนหน่วยความจำจริงที่ถูกใช้ ซึ่งภาพนี้อยู่ที่ 44%), **Commit Charge** (สัดส่วนของ commit charge เทียบกับ commit limit ซึ่งคือ RAM บวกขนาด pagefile.sys) และ **Hard Faults/sec** กราฟสองช่องแรกมีประโยชน์ต่างกันคือ ช่องแรกบอกว่ RAM เต็มหรือยัง ส่วนช่องที่สองบอกว่ระบบใกล้ชนเพดานการจองหรือยัง ซึ่งอาจชนได้แม้ RAM ยังเหลือ

3.3 สรุปตัวเลขทั้งหมดในส่วน Physical Memory

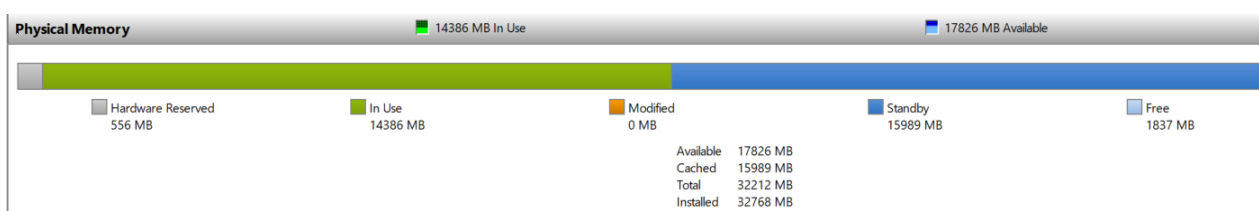


Figure 22: ส่วน Physical Memory ขยายให้เห็นตัวเลขทุกค่า

Table 7: ความหมายของแต่ละส่วนในแถบ Physical Memory

ส่วน	ค่าที่วัดได้	ความหมาย
Hardware Reserved	556 MB	หน่วยความจำที่ firmware และ hardware กันไว้ใช้เอง ระบบปฏิบัติการนำไปใช้ไม่ได้
In Use	14,386 MB	หน่วยความจำที่อยู่ใน working set ของ process, driver และ kernel จริง ณ ขณะนั้น
Modified	0 MB	page ที่ถูกแก้ไขแล้วและรอเขียนกลับลง disk ก่อนจึงจะนำไปใช้ใหม่ได้
Standby	15,989 MB	page ที่ถูกตัดออกจาก working set แล้วแต่ยังเก็บเนื้อหาไว้ ทำหน้าที่เป็น cache ของทั้งระบบ ดึงกลับได้ด้วย soft fault
Free	1,837 MB	page ว่างที่พร้อมแจกจ่ายทันที
Available	17,826 MB	หน่วยความจำที่ process ใหม่ขอใช้ได้ทันที เท่ากับ Standby บวก Free
Cached	15,989 MB	หน่วยความจำที่ทำหน้าที่เป็น cache เท่ากับ Standby บวก Modified
Total	32,212 MB	หน่วยความจำที่ระบบปฏิบัติการใช้งานได้ คือ Installed ลบ Hardware Reserved
Installed	32,768 MB	ขนาด RAM ที่ติดตั้งจริงบนเครื่อง (32 GB)

ย่อหน้าสรุปตัวเลขทั้งหมด เครื่องที่ใช้ทดลองติดตั้ง RAM ไว้ 32,768 MB หรือ 32 GB ในจำนวนนี้ถูก hardware กันไว้เป็น Hardware Reserved 556 MB เหลือให้ระบบปฏิบัติการใช้งานจริง 32,212 MB ตามที่บรรทัด Total รายงาน หน่วยความจำส่วนนี้ถูกแบ่งตามสถานะออกเป็นสี่ก้อน ได้แก่ In Use 14,386 MB ซึ่งเป็นส่วนที่ process และ kernel ถือครองอยู่จริง Modified 0 MB ซึ่งหมายความว่าไม่มี page ใดค้างรอเขียนกลับ disk เลยในขณะนั้น Standby 15,989 MB ซึ่งเป็นก้อนที่ใหญ่ที่สุดและทำหน้าที่เป็น cache ของทั้งระบบ และ Free 1,837 MB ที่ว่างพร้อมใช้งานทันที เมื่อรวมสี่ก้อนนี้เข้าด้วยกันจะได้ 32,212 MB เท่ากับค่า Total พอดี ส่วนค่าที่ระบบสรุปให้เพิ่มอีกสองค่าคือ Available 17,826 MB ซึ่งเท่ากับ Standby บวก Free และ Cached 15,989 MB ซึ่งเท่ากับ Standby บวก Modified ด้านบนของหน้าต่างยังรายงานสอดคล้องกันว่ามี 14,386 MB In Use และ 17,826 MB Available พร้อมกับตัวเลข 44% Used Physical Memory ซึ่งตรวจสอบได้จาก 14,386 หารด้วย 32,212 เท่ากับ 44.7% โดยประมาณ ภาพรวมทั้งหมดชี้ว่าเครื่องนี้ใช้หน่วยความจำไปไม่ถึงครึ่ง ยังมีหน่วยความจำพร้อมใช้อีกกว่า 17 GB และค่า Hard Faults/sec ของทุก process เป็นศูนย์ จึงไม่มีอาการขาดแคลนหน่วยความจำใด ๆ

จุดที่มักเข้าใจผิด ค่า Free เพียง 1,837 MB อาจทำให้เข้าใจว่าเครื่องเหลือหน่วยความจำน้อย แต่ความจริงแล้วหน่วยความจำที่ขอใช้ได้คือ Available 17,826 MB เพราะ Standby ทั้งก่อนสามารถถูกยึดคืนไปให้ process ใหม่ได้ทันทีเมื่อจำเป็น แนวคิดนี้ตรงกับความสัมพันธ์ระหว่าง free กับ available ของคำสั่ง free บน Linux ที่อธิบายไว้ในข้อ 2 ทุกประการ

3.4 การเทียบศัพท์ระหว่าง Windows กับ Linux

เมื่อนำข้อมูลจากข้อ 2 และข้อ 3 มาวางเทียบกัน จะเห็นว่าทั้งสองระบบรายงานสิ่งเดียวกันด้วยชื่อที่ต่างกัน

Table 8: การเทียบศัพท์ของเครื่องมือ monitoring ระหว่าง Windows และ Linux

Windows (Resource Monitor)	Linux (free / vmstat)	สิ่งที่วัดเหมือนกัน
Working Set	RSS (ps), used	หน่วยความจำที่ถือครองอยู่ใน RAM จริง
Private	used เฉพาะของ process	ส่วนที่ได้คืนแน่นอนเมื่อปิด process
Shareable	shared library mapping	ส่วนที่ใช้ร่วมกับ process อื่น
Standby + Modified (Cached)	buff/cache	หน่วยความจำที่ระบบยืมไปทำ cache
Available	available	หน่วยความจำที่ขอใช้ได้จริงโดยไม่ต้อง swap
Free	free	page ที่ยังไม่ถูกแตะต้องเลย
Commit Charge / Commit Limit	Committed_AS ใน /proc/meminfo	ปริมาณ virtual memory ที่ถูกจองไว้
Hard Faults/sec	si และ so ของ vmstat	การอ่านเขียน page กับ disk เพราะขาดหน่วยความจำ
pagefile.sys	swap partition หรือ swap file	พื้นที่บน disk ที่ใช้พัก page
Modified Page Writer	kswapd	ผู้ทำหน้าที่เขียน page กลับลง disk

ข้อสังเกตที่เชื่อมสองฝั่งเข้าด้วยกัน เครื่องที่ใช้ทดลองมี RAM 32 GB ตามที่ Resource Monitor รายงาน ขณะที่ Ubuntu บน WSL2 ในข้อ 2 มองเห็นหน่วยความจำเพียง 15 GiB (ราว 16 GB) เพราะ WSL2 ทำงานเป็น virtual machine ที่ได้รับส่วนแบ่งหน่วยความจำมาประมาณครึ่งหนึ่งของเครื่องตามค่าเริ่มต้น ตัวเลขทั้งสองฝั่งจึงไม่ขัดแย้งกัน แต่เป็นการมองหน่วยความจำคนละชั้นกัน คือฝั่ง Windows มองหน่วยความจำจริงของ hardware ส่วนฝั่ง Linux มองหน่วยความจำที่ virtual machine ได้รับมา

สรุปผลการทดลอง

ปฏิบัติการนี้เริ่มจากการทำความเข้าใจว่า virtual memory คือกลไกที่ระบบปฏิบัติการใช้แยก address space ของแต่ละ process ออกจากกัน และใช้ paging ร่วมกับ backing store บน disk เพื่อให้ใช้หน่วยความจำได้มากกว่าขนาด RAM จริง แม้ Linux และ MS Windows จะตั้งอยู่บนหลักการและ hardware เดียวกัน แต่ต่างกันที่วิธีจัดการ คือ Linux มองหน่วยความจำเป็น pool กลางของทั้งระบบและเรียกคืนตาม LRU ขณะที่ Windows จัดสรรเป็น working set รายบ้านแล้วรวบ page ที่ตัดออกไว้ในรายการกลาง และต่างกันที่นโยบายการจองคือ Linux ยอมให้ overcommit แล้วใช้ OOM killer เป็นทางออกสุดท้าย ส่วน Windows กันโควตาด้วย commit charge ตั้งแต่ตอนจอง

การทดลองเพิ่มและลด swap ด้วย swap file สำเร็จครบทุกขั้นตอน และยังทำให้พบข้อจำกัดที่ไม่ได้เขียนไว้ในคู่มือทั่วไป คือบน WSL2 ซึ่ง systemd ตรวจพบว่าตัวเองทำงานอยู่ใน container นั้น systemd-fstab-generator จะข้าม swap entry ใน /etc/fstab ทั้งทั้งหมด การทำให้ swap ถาวรบน WSL2 จึงต้องใช้ swapon -a ผ่าน /etc/wsl.conf หรือกำหนดขนาด swap จากฝั่ง Windows ด้วย .wslconfig แทน

ด้านเครื่องมือเฝ้าดู free ตอบคำถามว่าหน่วยความจำเหลือเท่าไรในขณะนั้น โดยมีค่า available เป็นตัวเลขที่ควรใช้ตัดสินใจมากกว่า free ส่วน vmstat ตอบคำถามว่าระบบกำลังเคลื่อนไหวอย่างไร โดยต้องระวังว่าแถวแรกเป็นค่าเฉลี่ยตั้งแต่บูต ไม่ใช่ค่าปัจจุบัน ฝั่ง Windows นั้น Resource Monitor ให้ข้อมูลระดับเดียวกันในรูปแบบกราฟิก และเมื่อเทียบศัพท์กันแล้วจะเห็นว่าทั้งสองระบบรายงานสิ่งเดียวกัน เช่น Standby ของ Windows ทำหน้าที่อย่างเดียวกับ buff/cache ของ Linux และค่า Available ของทั้งสองระบบมีความหมายตรงกันทุกประการ

บทเรียนร่วมที่ได้จากทั้งสามข้อคือ **หน่วยความจำที่ดูเหมือนถูกใช้ไปไม่ได้หมายความว่าหายไป** ทั้ง Linux และ Windows ต่างยืมหน่วยความจำว่างไปทำ cache เพื่อเร่งความเร็วของระบบ และคืนให้ทันทีที่ process ต้องการ การอ่านค่าจากเครื่องมือ monitoring จึงต้องดูที่ค่า available หรือ Available เป็นหลัก ไม่ใช่ค่า free เพียงอย่าง

เดี่ยว

เอกสารอ้างอิง

1. `man 1 free`, `man 8 vmstat`, `man 8 swapon`, `man 8 mkswap` และ `man 5 proc` จาก package `procps-ng 4.0.4` และ `util-linux` บน Ubuntu 24.04.1
2. The Linux Kernel Documentation, *Memory Management* และ *Documentation/admin-guide/sysctl/vm.rst* (คำอธิบาย `vm.swappiness` และ `vm.overcommit_memory`)
3. Russinovich, M., Solomon, D., Ionescu, A., Yosifovich, P., *Windows Internals, Part 1*, 7th Edition, Microsoft Press (บท Memory Management: working set, VAD, PFN database, commit charge)
4. Microsoft Learn, *Resource Monitor* และ *Windows Subsystem for Linux: Advanced settings configuration* (`wsl.conf` และ `.wslconfig`)
5. systemd documentation, `systemd-fstab-generator(8)` (เงื่อนไขการข้าม swap entry เมื่อทำงานใน container)